

Avaliação Parcial 02 — Desenvolvimento de uma API RESTful

1. Contextualização

Uma instituição de ensino deseja disponibilizar um sistema para gerenciamento de seus eventos acadêmicos, como palestras, workshops, minicursos, seminários e outras atividades.

Para isso, você deverá desenvolver uma **API RESTful utilizando Python e FastAPI**, responsável por disponibilizar os recursos necessários para o gerenciamento dos eventos e de seus participantes.

A API deverá seguir os princípios apresentados durante as aulas da disciplina, utilizando corretamente os métodos HTTP, códigos de status, JSON, validação de dados e separação de responsabilidades por meio da arquitetura MVC.

2. Objetivo

Desenvolver uma **API RESTful completa**, utilizando:

- Python;
- FastAPI;
- Pydantic;
- JSON;
- Métodos HTTP;
- Arquitetura MVC;
- Validação de dados;
- Tratamento de erros;
- Status HTTP apropriados.

A aplicação deverá permitir que um cliente, como um Front-End ou ferramenta como o Swagger, possa realizar operações sobre os recursos da API.

3. Requisitos funcionais

A API deverá possuir, no mínimo, os seguintes recursos:

3.1 Eventos

Cada evento deverá possuir, no mínimo:

- `id`
- `titulo`
- `descricao`
- `data`
- `horario`
- `local`
- `capacidade`
- `categoria`

Exemplos de categorias:

- Palestra
- Workshop
- Minicurso
- Seminário
- Competição

A API deverá permitir:

Cadastrar evento

POST `/eventos`

Deverá receber os dados do evento em formato JSON e retornar o evento cadastrado.

Listar eventos

GET `/eventos`

Deverá retornar a lista de eventos cadastrados.

Consultar evento

GET `/eventos/{id}`

Deverá retornar os dados de um evento específico.

Caso o evento não exista, a API deverá retornar um código HTTP adequado e uma mensagem informando o problema.

Atualizar evento

PUT `/eventos/{id}`

Deverá permitir a atualização dos dados de um evento existente.

Excluir evento

DELETE `/eventos/{id}`

Deverá remover um evento existente.

4. Participantes

Cada participante deverá possuir, no mínimo:

- `id`
- `nome`
- `email`
- `curso`

A API deverá permitir:

Cadastrar participante:

POST `/participantes`

Listar participantes:

GET `/participantes`

Consultar participante:

GET `/participantes/{id}`

Atualizar participante:

PUT `/participantes/{id}`

Excluir participante:

DELETE `/participantes/{id}`

5. Inscrições

Além dos recursos anteriores, a API deverá permitir que um participante se inscreva em um evento.

Para isso, deverá existir uma rota semelhante a:

POST `/eventos/{evento_id}/inscricoes/{participante_id}`

A API deverá verificar, no mínimo:

1. Se o evento existe;
2. Se o participante existe;
3. Se o participante já está inscrito no evento;
4. Se o evento ainda possui vagas disponíveis.

Caso alguma dessas condições não seja atendida, a API deverá retornar um **código HTTP adequado** e uma mensagem explicando o erro.

Também deverá existir uma forma de consultar os participantes inscritos em determinado evento:

GET `/eventos/{evento_id}/inscricoes`

6. Validação dos dados

A aplicação deverá utilizar os recursos de validação do **Pydantic/FastAPI**.

Exemplos de regras que deverão ser implementadas:

- O título do evento não pode ser vazio;
- O nome do participante não pode ser vazio;
- O e-mail deve possuir formato válido;
- A capacidade do evento deve ser maior que zero;
- A data do evento deve ser válida;
- Um evento não pode possuir capacidade negativa;
- Os campos obrigatórios devem ser informados.

Os alunos deverão testar também situações em que os dados enviados são inválidos.

7. Tratamento de erros

A API deverá tratar situações de erro de forma adequada.

Por exemplo:

Evento inexistente

```
{  
  "detail": "Evento não encontrado."  
}
```

Participante já inscrito

```
{  
  "detail": "Participante já está inscrito neste evento."  
}
```

Evento sem vagas

```
{  
  "detail": "Não existem vagas disponíveis para este evento."  
}
```

Os códigos HTTP deverão ser utilizados de acordo com a situação.

Exemplos:

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 404 Not Found
- 422 Unprocessable Entity

8. Arquitetura MVC

O projeto deverá utilizar uma estrutura que demonstre a separação de responsabilidades apresentada em aula.

Uma sugestão de organização é:

```
projeto/
|
├── main.py
|
├── controller/
|   ├── evento_controller.py
|   └── participante_controller.py
|
├── model/
|   ├── evento.py
|   └── participante.py
|
└── service/
    ├── evento_service.py
    └── participante_service.py
```

Os nomes e a organização dos arquivos podem ser adaptados pela equipe, desde que fique clara a separação entre as responsabilidades da aplicação.

Controller

Responsável pelas rotas e pelo recebimento das requisições HTTP.

Model

Responsável pela representação e validação dos dados da aplicação.

Service

Responsável pelas regras de negócio.

Por exemplo:

Verificar se ainda existem vagas antes de realizar uma inscrição.

Essa regra não deve ficar diretamente misturada com a definição da rota.

9. Armazenamento dos dados

Nesta primeira versão da APS, **não é obrigatório utilizar banco de dados**.

Os dados poderão ser armazenados em estruturas em memória, como listas e dicionários Python.

Exemplo:

```
eventos = []
participantes = []
inscricoes = []
```

O objetivo principal desta atividade é avaliar os conceitos de **Back-End, HTTP, REST, FastAPI, validação e arquitetura MVC** apresentados até o momento.

10. Documentação e testes

A API deverá ser testada utilizando o **Swagger disponibilizado automaticamente pelo FastAPI**.

Os alunos deverão demonstrar o funcionamento das principais operações:

- Cadastro;
- Consulta;
- Atualização;
- Exclusão;
- Inscrição em evento;
- Tratamento de situações de erro.

Também deverão ser apresentados exemplos de requisições e respostas em JSON.

11. Requisitos mínimos

Para que o projeto seja considerado completo, deverá possuir:

- API desenvolvida com FastAPI;
- Pelo menos dois recursos principais;
- Operações GET, POST, PUT e DELETE;
- Uso adequado de códigos HTTP;
- Entrada e saída de dados em JSON;

- Modelos utilizando Pydantic;
- Validação dos dados;
- Tratamento de erros;
- Regra de negócio envolvendo os recursos;
- Separação de responsabilidades utilizando MVC;
- Documentação e testes através do Swagger;
- Código organizado e legível.

12. Entrega

A equipe deverá entregar:

1. Código-fonte completo do projeto;
2. Arquivo `requirements.txt`;
3. `README.md` contendo:
 - descrição do projeto;
 - instruções para instalação;
 - instruções para execução;
 - descrição das principais rotas;
 - exemplos de utilização;
4. Link para o repositório no GitHub;